# Vision Document

## Restaurant Operations System (MVP – SaaS Ready)

## 1. System Vision

The system is a cloud-based restaurant operations platform designed to digitalize order management for a single restaurant (MVP version).

Although the initial implementation will support only one restaurant (single-tenant), the architecture will be designed to allow future evolution into a multi-tenant SaaS platform.

The system will centralize:

- Internal order management (tables and counter)
- External web orders (delivery / pickup)
- Order state transitions
- Sales closure and integration preparation for Siigo
- Operational notifications via WhatsApp

---

## 2. Problem Statement

Small and medium restaurants often manage orders manually or using fragmented tools (WhatsApp, paper notes, Excel).

This causes:

- Order tracking errors
- Lack of traceability
- Poor visibility of operational flow
- No structured integration with accounting systems

---

## 3. Users and Actors

### Internal Users

- **Administrator / Owner**
- **Cashier**
- **Waiter**

### External Actors

- **Customer (web ordering)**
- **Siigo API (future integration)**
- **WhatsApp API (notifications)**

Kitchen staff and delivery drivers will not access the system directly.
Operational flow will reach them via order printing/display and WhatsApp notifications.

---

# 4. MVP Scope (Single-Tenant)

## Included

- Authentication and role-based access control (RBAC)
- Restaurant configuration (single instance)
- Menu management (products, categories)
- Table management
- Order creation (internal and web)
- Order state lifecycle:
  - CREATED
  - IN_PREPARATION
  - READY
  - OUT_FOR_DELIVERY
  - DELIVERED
  - CLOSED
- Sales closure
- Preparation of accounting payload for Siigo (future adapter)
- WhatsApp notification trigger
- Basic operational dashboard
- Audit fields (created_at, updated_at, created_by)

## Excluded (for MVP)

- Multi-tenant support
- Native mobile apps
- Built-in accounting system
- Online payment gateway
- Inventory management
- Direct kitchen/delivery app

---

# 5. Non-Functional Requirements

## Security

- JWT authentication
- Password hashing
- Role-based authorization
- Input validation
- Environment-based secrets
- CORS configuration

## Availability

- Health check endpoint
- Dockerized deployment
- Database backups (AWS RDS planned)

### Observability

- Structured logging
- Order traceability
- Error handling standardization

### Performance

- Order creation < 500ms (target p95)
- State transition operations < 300ms

---

## 6. Architectural Strategy

The system will be implemented as a:

> Modular Monolith using Clean Architecture principles.

Layers:

- Domain
- Application (Use Cases)
- Infrastructure (Prisma, external adapters)
- Interface (Controllers)

The Domain layer will not depend on frameworks.

---

## 7. Future Evolution (Phase 2 – SaaS)

The system is designed to evolve into a multi-tenant SaaS platform by:

- Introducing tenant_id in all core entities
- Isolating data per tenant
- Adding subscription plans
- Enabling multi-restaurant management

This is intentionally excluded from MVP to reduce complexity and ensure delivery quality.

---

## 8. Success Metrics

- Stable production deployment in AWS
- End-to-end order lifecycle without data inconsistency
- Integration-ready architecture
- Clean separation of responsibilities (SOLID compliance)